РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ

ФАКУЛЬТЕТ ФИЗИКО-МАТЕМАТИЧЕСКИХ И ЕСТЕСТВЕННЫХ НАУК

Лабораторная работа

Дубохань НКАбд-04-25

Москва

2026

Цель работы

Изучить основы программирования в оболочке ОС UNIX/Linux. Научиться писать небольшие командные файлы

Последовательность выполнения работы

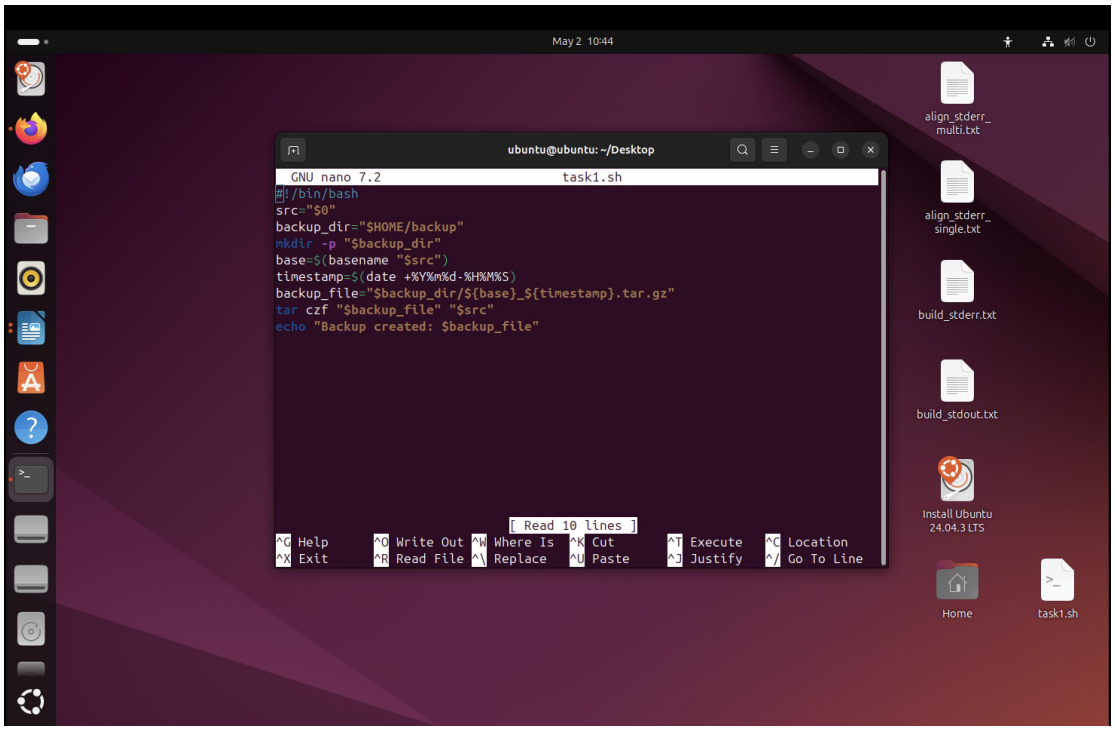
1. Написать скрипт, который при запуске будет делать резервную копию самого себя (то есть файла, в котором содержится его исходный код) в другую директорию backup в вашем домашнем каталоге. При этом файл должен архивироваться одним из ар-хиваторов на выбор zip, bzip2 или tar. Способ использования команд архивации необходимо узнать, изучив справку.

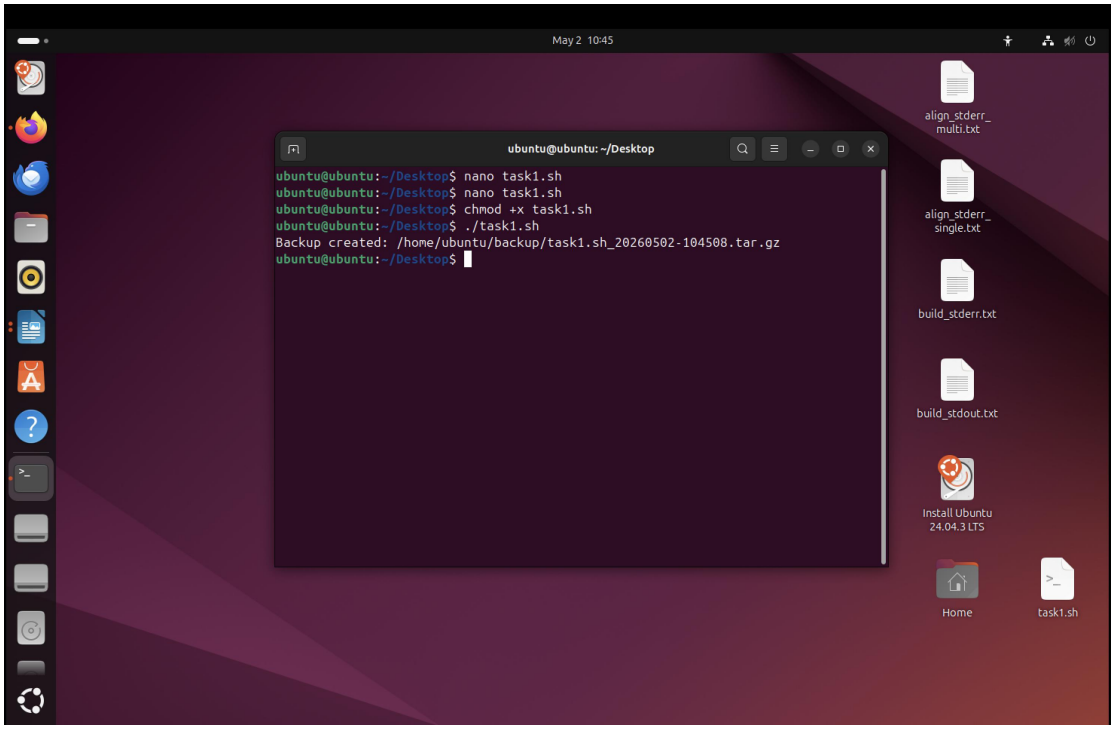
2. Написать пример командного файла, обрабатывающего любое произвольное число аргументов командной строки, в том числе превышающее десять. Например, скрипт может последовательно распечатывать значения всех переданных аргументов.

3. Написать командный файл — аналог команды Is (без использования самой этой команды и команды dir). Требуется, чтобы он выдавал информацию о нужном каталоге и выводил информацию о возможностях доступа к файлам этого каталога.

4. Написать командный файл, который получает в качестве аргумента командной строки формат файла (.txt, .doc, jpg, . pdf и т.д.) и вычисляет количество таких файлов в указанной директории. Путь к директории также передаётся в виде аргумента командной строки.

1. Написать скрипт, который при запуске будет делать резервную копию самого себя (то есть файла, в котором содержится его исходный код) в другую директорию backup в вашем домашнем каталоге. При этом файл должен архивироваться одним из ар-хиваторов на выбор zip, bzip2 или tar. Способ использования команд архивации необходимо узнать, изучив справку.





2. Написать пример командного файла, обрабатывающего любое произвольное число аргументов командной строки, в том числе превышающее десять. Например, скрипт может последовательно распечатывать значения всех переданных аргументов.

```
ubuntu@ubuntu: ~/Desktop
  GNU nano 7.2                    task2.sh *
#!/bin/bash
echo "Total parameters: $#"
echo "All parameters:"
count=1
for arg in "$@"; do
    echo "  Arg $count: $arg"
    ((count++))
done
if [ $# -ge 10 ]; then
    echo "The 10th argument is: ${10}"
fi
```

```
ubuntu@ubuntu:~$ chmod +x task2.sh
chmod: cannot access 'task2.sh': No such file or directory
ubuntu@ubuntu:~$ nano task2.sh
ubuntu@ubuntu:~$ chmod +x task2.sh
ubuntu@ubuntu:~$ ls -l task2.sh
-rwxrwxr-x 1 ubuntu ubuntu 207 May  2 10:51 task2.sh
ubuntu@ubuntu:~$
```

```
ubuntu@ubuntu:~$ nano task2.sh
ubuntu@ubuntu:~$ chmod +x task2.sh
ubuntu@ubuntu:~$ ls -l task2.sh
-rwxrwxr-x 1 ubuntu ubuntu 207 May  2 10:51 task2.sh
ubuntu@ubuntu:~$ ./task2.sh a b c d e f g h i j k l m n o
Total parameters: 15
All parameters:
  Arg 1: a
  Arg 2: b
  Arg 3: c
  Arg 4: d
  Arg 5: e
  Arg 6: f
  Arg 7: g
  Arg 8: h
  Arg 9: i
  Arg 10: j
  Arg 11: k
  Arg 12: l
  Arg 13: m
  Arg 14: n
  Arg 15: o
The 10th argument is: j
ubuntu@ubuntu:~$
```

3. Написать командный файл — аналог команды Is (без использования самой этой команды и команды dir). Требуется, чтобы он выдавал информацию о нужном каталоге и выводил информацию о возможностях доступа к файлам этого каталога.

```
ubuntu@ubuntu: ~/Desktop
GNU nano 7.2                    task3.sh *
#!/bin/bash
target="${1:-.}"
if [ ! -d "$target" ]; then
    echo "Error: '$target' is not a directory."
    exit 1
fi
echo "Contents of directory: $target"
echo "================================"
for file in "$target"/*; do
    if [ -e "$file" ]; then
        perm=$(stat -c "%A" "$file" 2>/dev/null || stat -f "%Sp" "$file" 2>/dev>
        echo "$perm  $(basename "$file")"
    fi
done
```

```
ubuntu@ubuntu:~/Desktop$ chmod +x task3.sh
ubuntu@ubuntu:~/Desktop$ ./task3.sh /tmp
Contents of directory: /tmp
================================
srwxrwxr-x  OSL_PIPE_1000_SingleOfficeIPC_0c2b9f29dd5c3e2fd44b51148e0a4bb
-rw-------  gdm3-config-err-M315N3
drwx------  lu100173a61x.tmp
drwx------  snap-private-tmp
drwx------  systemd-private-1b9c4706ac0d4537b13be6db26d9ad6f-ModemManager.servic
e-L7bwNV
drwx------  systemd-private-1b9c4706ac0d4537b13be6db26d9ad6f-colord.service-pps3
w6
drwx------  systemd-private-1b9c4706ac0d4537b13be6db26d9ad6f-fwupd.service-zVZBc
3
drwx------  systemd-private-1b9c4706ac0d4537b13be6db26d9ad6f-polkit.service-BMKx
2i
drwx------  systemd-private-1b9c4706ac0d4537b13be6db26d9ad6f-power-profiles-daem
on.service-XbQx5F
drwx------  systemd-private-1b9c4706ac0d4537b13be6db26d9ad6f-spice-vdagentd.serv
ice-Lhnk2i
drwx------  systemd-private-1b9c4706ac0d4537b13be6db26d9ad6f-switcheroo-control.
service-ec355V
drwx------  systemd-private-1b9c4706ac0d4537b13be6db26d9ad6f-systemd-logind.serv
```

4. Написать командный файл, который получает в качестве аргумента командной строки формат файла (.txt, .doc, jpg, . pdf и т.д.) и вычисляет количество таких файлов в указанной директории. Путь к директории также передаётся в виде аргумента командной строки.

```
ubuntu@ubuntu: ~/Desktop
  GNU nano 7.2                    task4.sh *
#!/bin/bash
if [ $# -ne 2 ]; then
    echo "Usage: $0 <extension> <directory>"
    echo "Example: $0 .txt /home/user/docs"
    exit 1
fi
ext="$1"
dir="$2"
[[ "$ext" != .* ]] && ext=".$ext"
if [ ! -d "$dir" ]; then
    echo "Error: Directory '$dir' does not exist."
    exit 1
fi
shopt -s nullglob
files=("$dir"/*"$ext")
count=${#files[@]}
echo "Number of files with extension '$ext' in '$dir': $count"
```

```
ubuntu@ubuntu:~/Desktop$ nano task4.sh
ubuntu@ubuntu:~/Desktop$ chmod +x task4.sh
ubuntu@ubuntu:~/Desktop$ ./tas4..sh /tmp
bash: ./tas4..sh: No such file or directory
ubuntu@ubuntu:~/Desktop$ ./task4.sh .txt /tmp
Number of files with extension '.txt' in '/tmp': 0
ubuntu@ubuntu:~/Desktop$
```

ВЫХОД

1. Создан скрипт для автоматического резервного копирования самого себя с использованием tar.

2. Реализована обработка произвольного числа аргументов командной строки (включая более 10) с их последовательным выводом.

3. Написан аналог команды ls без использования ls/dir – вывод прав доступа и имён файлов через stat.

4. Разработан скрипт для подсчёта файлов с заданным расширением в указанной директории.

В результате освоены основы программирования на Bash: работа с переменными, позиционными параметрами, циклами, условиями, file tests и командами tar, stat. Полученные навыки применимы для автоматизации задач администрирования в UNIX/Linux.

Контрольные вопросы

1. Объясните понятие командной оболочки. Приведите примеры командных оболочек. Чем они отличаются?

2. Что такое POSIX?

3. Как определяются переменные и массивы в языке программирования bash?

4. Каково назначение операторов let и read?

5. Какие арифметические операции можно применять в языке программирования bash?

6.Что означает операция (( ))?

7. Какие стандартные имена переменных Вам известны?

8. Что такое метасимволы?

9. Как экранировать метасимволы?

10. Как создавать и запускать командные файлы?

11. Как определяются функции в языке программирования bash?

12. Каким образом можно выяснить, является файл каталогом или обычным файлом?

13. Каково назначение команд set, typeset и unset?

14. Как передаются параметры в командные файлы?

15. Назовите специальные переменные языка bash и их назначение.

1. Командная оболочка (shell) — программа, обеспечивающая взаимодействие пользователя с ОС. Примеры: sh (Bourne), bash (Bourne Again), csh, ksh. Отличаются синтаксисом, возможностями (история, автодополнение, арифметика, массивы).

2. POSIX (Portable Operating System Interface) — набор стандартов IEEE, обеспечивающих совместимость UNIX-подобных систем и переносимость приложений на уровне исходного кода.

3. Переменные: var=значение (без пробелов). Использование: $var. Массивы: arr=(a b c); доступ ${arr[0]}; все элементы ${arr[@]}.

4. let — выполнение арифметических операций (например, let x=5+3). read — чтение ввода пользователя в переменные: read var.

5. Арифметические операции: + - * / % ** (умножение, деление, остаток, возведение в степень), сравнения < > <= >= == !=, логические && || !, битовые & | ^ << >>, присваивания = += -= *= /= %= <<= >>= &= ^= |=.

6. (( )) — двойные скобки для арифметических вычислений и условий. Внутри не нужен $ для переменных. Пример: ((x = 5 + 3)) или while ((x > 0)).

7. Стандартные переменные: PATH, HOME, USER, SHELL, PS1, PS2, IFS, MAIL, TERM, PWD, OLDPWD, RANDOM, SECONDS.

8. Метасимволы — символы, имеющие специальное значение в shell: * ? [ ] { } ( ) < > | & ; ! $ \ " ' и пробел, табуляция, новая строка (IFS).

9. Экранирование — отмена специального значения:

обратный слэш *

одинарные кавычки '*' (полное экранирование)

двойные кавычки "*" (экранируют все, кроме $, `, \, ").

10. Создание: текстовый файл с первой строкой #!/bin/bash. Запуск: сделать исполняемым chmod +x file, затем ./file или bash file.

11. Функции:

```bash
function имя { команды; }
# или
имя() { команды; }
```

12. Использовать команду test или [ ]:

[-d файл] — истина, если каталог.

[-f файл] — истина, если обычный файл.

13. set — установка параметров shell / позиционных параметров.

typeset (синоним declare) — задание атрибутов переменных (-i целое, -r только чтение, -f функции). unset — удаление переменных или функций (-f).

14. Параметры передаются после имени скрипта: ./script arg1 arg2. Внутри доступны как $1, $2, …, $9, ${10} и т.д. $# — количество, $@ или $* — все параметры.

15. Специальные переменные:

$0 — имя скрипта

$1…$9 — позиционные параметры

$# — число параметров

$@, $* — все параметры (разное поведение с кавычками)

$? — код возврата последней команды

$$ — PID текущего shell

$! — PID последней фоновой команды

$- — текущие опции shell

$_ — последний аргумент предыдущей команды.